

1
Week2
Day5
TopicsClosures & HOF
Focus

Topic 1 — Lexical Scope & Closure Mechanics

Lexical Scope ka matlab hai: ek function ka scope wahan decide hota hai jahan wo **define** hua hai — na ki jahan se call hua. JavaScript mein inner functions outer functions ke variables access kar sakte hain, lekin outer functions inner ke variables nahi dekh sakte.

Closure tab banta hai jab ek function apne outer scope ke variables ko yaad rakhta hai — even after outer function execute ho chuka ho. Ye JavaScript ka sabse powerful aur confusing feature hai.

Lexical Scope

Function ka scope uski definition location se decide hota hai, call location se nahi.

Closure

Inner function outer function ke variables ko 'close over' karta hai — memory mein retain hote hain.

Scope Chain

JS pehle local scope mein variable dhundta hai, phir outer, phir global — chain follow karta hai.

Basic Lexical Scope Example

```
// Basic Lexical Scope
const globalVar = 'global';

function outer() {
  const outerVar = 'outer';

  function inner() {
    const innerVar = 'inner';
    console.log(innerVar); // ■ 'inner' – own scope
    console.log(outerVar); // ■ 'outer' – parent scope (lexical)
    console.log(globalVar); // ■ 'global' – global scope
  }

  inner();
  // console.log(innerVar); – ■ ReferenceError! outer can't see inner
}

outer();
```

Closure — Counter Factory (Real World Pattern)

```
// ■ Classic Closure – counter factory
function makeCounter(start = 0) {
  let count = start; // Private variable – only accessible via closure
  return {
    increment: () => ++count,
    decrement: () => --count,
    reset: () => { count = start; },
  };
}
```

```

getCount: () => count
};
}

const counterA = makeCounter(10);
const counterB = makeCounter(0); // Independent closure!

counterA.increment(); // 11
counterA.increment(); // 12
counterB.increment(); // 1 – counterB has its OWN count
counterA.getCount(); // 12 – counterA unaffected

// 'count' is NOT accessible from outside:
// counterA.count – undefined

```

■ Critical Trap — Closure in Loops (var vs let)

Ye interview mein 100% poochha jaata hai. var se loop closure bug hota hai.

Reason: var function-scoped hai — sab iterations SAME variable share karte hain.

Fix: let use karo (block-scoped — har iteration ka apna variable) ya IIFE wrap karo.

■ Bug

```

// ■ BUG – var in loop closure
for (var i = 0; i < 3; i++) {
  setTimeout(function() {
    console.log(i);
  }, 100);
}

// Output: 3, 3, 3 (NOT 0,1,2!)
// Why: var i is shared,
// loop ends before timeout,
// i = 3 by then

```

■ Fix

```

// ■ FIX 1 – use let
for (let i = 0; i < 3; i++) {
  setTimeout(function() {
    console.log(i);
  }, 100);
}

// Output: 0, 1, 2 ■

// ■ FIX 2 – IIFE
for (var i = 0; i < 3; i++) {
  (function(j) {
    setTimeout(() => console.log(j), 100);
  })(i);
}

```

Closure for Data Privacy — Bank Account Module

```

// ■ Closure for DATA PRIVACY – Module Pattern
function createBankAccount(initialBalance) {
  let balance = initialBalance; // Private – no direct access!
  const transactions = []; // Private history
  return {
    deposit(amount) {
      if (amount <= 0) throw new Error('Invalid amount');
      balance += amount;
      transactions.push({ type: 'credit', amount });
    },
    withdraw(amount) {
      if (amount > balance) throw new Error('Insufficient funds');
      balance -= amount;
    }
  };
}

```

```

transactions.push({ type: 'debit', amount });
},
getBalance: () => balance,
getHistory: () => [...transactions], // spread – can't mutate original
};
}

const acc = createBankAccount(1000);
acc.deposit(500);
acc.withdraw(200);
acc.getBalance(); // 1300
acc.balance; // undefined – truly private!

```

Topic 2 — IIFE (Immediately Invoked Function Expression)

IIFE ek function hai jo define hone ke turant baad execute ho jaata hai. Ye global scope ko pollute karne se bachata hai aur ek private scope create karta hai. ES6 se pehle ye bahut common tha — aaj bhi kuch patterns mein useful hai.

IIFE Patterns

```

// Basic IIFE syntax
(function() {
  const secret = 'I am private';
  console.log('IIFE ran!');
})(); // <-- () yahan immediately call kar raha hai

// Arrow function IIFE
(() => {
  console.log('Arrow IIFE!');
})();

// IIFE with parameters
((name, role) => {
  console.log(`${name} is a ${role}`);
})('Rahul', 'Admin'); // 'Rahul is a Admin'

// IIFE returning a value
const config = (() => {
  const env = 'production';
  const port = env === 'production' ? 443 : 3000;
  return { env, port, debug: env !== 'production' };
})();
console.log(config); // { env: 'production', port: 443, debug: false }

```

■ Real-World IIFE Use

jQuery plugins mein \$ conflict avoid karne ke liye: `(function($) { ... })(jQuery);`

Initialization code jo ek baar run ho aur global scope pollute na kare.

ES Modules aane ke baad IIFE ki zaroorat kam ho gayi hai — lekin legacy code mein aaj bhi milta hai.

Topic 3 — map(), filter(), reduce() Deep-Dive

Ye teeno **Higher-Order Functions (HOF)** hain jo ek function argument ke roop mein lete hain. Ye immutable hote hain — original array modify nahi karte. Functional programming ka core hain.

map()	filter()	reduce()
Har element pe function apply karo — SAME length ka naya array milta hai.	Condition ke basis pe elements rakho — SMALLER ya same length ka array milta hai.	Pura array ek value mein 'reduce' karo — number, string, object kuch bhi.

map() — Transform Every Element

```
// ■■■ MAP ■■■  
const students = [  
  { name: 'Rahul', marks: [80, 90, 85] },  
  { name: 'Priya', marks: [95, 88, 92] },  
  { name: 'Arjun', marks: [60, 72, 68] },  
  { name: 'Sneha', marks: [78, 85, 90] },  
];  
  
// Transform: add average to each student  
const withAvg = students.map(s => ({  
  ...s,  
  avg: (s.marks.reduce((a, b) => a + b, 0) / s.marks.length).toFixed(1)  
}));  
  
// [{ name:'Rahul', marks:[...], avg:'85.0' }, ...]  
  
// Extract just names  
const names = students.map(s => s.name);  
// ['Rahul', 'Priya', 'Arjun', 'Sneha']  
  
// Convert marks to letter grades  
const graded = students.map(s => {  
  const avg = s.marks.reduce((a,b)=>a+b,0) / s.marks.length;  
  return { name: s.name, grade: avg>=90?'A':avg>=80?'A':avg>=70?'B':'C' };  
});
```

filter() — Select Elements by Condition + Common Mistake

[illegible]

```
const correct = students
  .filter(s => s.marks[0] > 70) // first filter
  .map(s => s.name); // then transform
// ['Rahul', 'Priya', 'Sneha']
```

reduce() — Accumulate to Any Shape + Advanced Patterns

```
// ■■■ REDUCE ■■■■
// Signature: array.reduce(callback(accumulator, current), initialValue)
// Simple: sum of numbers
const nums = [10, 20, 30, 40];
const total = nums.reduce((acc, n) => acc + n, 0); // 100
// Max value (no Math.max needed)
const max = nums.reduce((acc, n) => n > acc ? n : acc, -Infinity); // 40
// Count occurrences – reduce to object
const fruits = ['apple','banana','apple','mango','banana','apple'];
const count = fruits.reduce((acc, fruit) => {
    acc[fruit] = (acc[fruit] || 0) + 1;
    return acc;
}, {});
// { apple: 3, banana: 2, mango: 1 }
// Group by – reduce to grouped object
const byGrade = students.reduce((acc, s) => {
    const avg = s.marks.reduce((a,b)=>a+b,0)/s.marks.length;
    const g = avg>=90?'A':avg>=80?'A':'B';
    acc[g] = acc[g] || [];
    acc[g].push(s.name);
    return acc;
}, {});
// { 'A+': ['Priya'], 'A': ['Rahul','Sneha'], 'B': ['Arjun'] }
// Flatten nested arrays using reduce
const nested = [[1,2],[3,4],[5,6]];
const flat = nested.reduce((acc, arr) => [...acc, ...arr], []);
// [1,2,3,4,5,6]
```

■ Critical: reduce() ke 3 Common Mistakes

1. initialValue miss karna — empty array pe crash karta hai: `[] .reduce((a,b)=>a+b) → TypeError!`
2. accumulator return na karna — acc undefined ho jaata hai aur sab kuch break.
3. Object accumulator mutate karna — return acc bhool jaana. Hamesha return karo!

Topic 4 — Function Composition & Currying

Function Composition ka matlab hai: chhote functions ko combine karke ek bada function banana. Output of one function → input of next. Unix pipe jaisa sochna. **Currying** ek technique hai jisme multi-argument function ko chain of single-argument functions mein convert karte hain.

Currying — Partial Application


```
);  
processStudents(students);  
// Sorted by avg with ranks assigned
```

Topic 5 — Memoization

Memoization ek optimization technique hai jisme function ke results cache karte hain. Agar same arguments se dobara call ho to cached result return karte hain — function dobara execute nahi hota. Ye closures ka ek classic real-world application hai.

Memoization — Cache with Closure

```
// Without memoization – expensive re-computation  
function fibonacci(n) {  
  if (n <= 1) return n;  
  return fibonacci(n - 1) + fibonacci(n - 2);  
}  
fibonacci(40); // Very slow! exponential time  $O(2^n)$   
  
// With memoization – cache results using closure  
function memoize(fn) {  
  const cache = {}; // Private cache via closure  
  return function(...args) {  
    const key = JSON.stringify(args);  
    if (cache[key] !== undefined) {  
      console.log(`Cache hit for ${key}`);  
      return cache[key];  
    }  
    cache[key] = fn.apply(this, args);  
    return cache[key];  
  };  
}  
  
const fastFib = memoize(function fib(n) {  
  if (n <= 1) return n;  
  return fastFib(n-1) + fastFib(n-2);  
});  
fastFib(40); // Fast!  $O(n)$  now  
fastFib(40); // Instant – cache hit!  
  
// Real-world: memoize API calls  
const fetchUser = memoize(async (userId) => {  
  const res = await fetch(`/api/users/${userId}`);  
  return res.json();  
});  
fetchUser(1); // API call karta hai  
fetchUser(1); // Cached – no API call!  
fetchUser(2); // New key – API call
```

■ Memoization vs Caching vs useMemo (React)

Memoization: Function level caching via closure — ye aaj ka topic.

React useMemo: React hook jo component mein expensive computations memoize karta hai.

Redis/HTTP Cache: Server/network level caching — alag concept hai.

Rule of thumb: Pure functions (same input = same output) hi memoize karo.

Day 2 Learning Outcomes

Closure kya hai aur kaise kaam karta hai — ye confidently explain kar paoge.

Closure loop bug identify aur fix kar paoge (interview favorite!).

map(), filter(), reduce() chaining mein use kar paoge without bugs.

Currying aur partial application se reusable functions bana paoge.

Memoization implement karke expensive functions optimize kar paoge.